RALLYSTATS · DOCUMENTO INTERNO

Funcionalidades pendientes: análisis de implementación

Las 10 funcionalidades del roadmap premium, ordenadas de la más fácil a la más difícil según el estado del código, con lo que se reusa, lo que falta construir, los riesgos y un boceto de cómo se vería cada una. Siete ya están hechas.

Basado en una revisión del código al 3 de octubre de 2026: `volley_stats_app` en `cc57d27` y `volley_stats_app_backend` en `f7b3c31`.

Complementa al *Roadmap de funcionalidades premium* (30/08/2026).

Los bocetos son ilustrativos (muestran ubicación y flujo, no un diseño final). Es un insumo de decisión, no un plan cerrado.

> **Actualizado el 24 de octubre de 2026:** la sección 3 (sync de partidos y equipos, Etapas 1 y 2) quedó implementada y probada en dos dispositivos el 23/10 y sale en la versión **1.3.0**, junto con la papelera, la política de privacidad 2.0 y el arreglo del inicio de sesión. Qué cambió al implementarla y cómo se probó está en **"3 · Cómo quedó"**, al final de la sección. La sección 8 se re-estimó con el sync ya hecho. Las funcionalidades 1, 2 y 4 (Tanda 1) se implementaron el 04/10/2026, en la versión 1.2.0; las 5, 6 y 7 después.

Contenido

A Qué cambió desde el roadmap del 30/08

B Tabla comparativa y esfuerzo relativo

1 Notas de scouting por partido · implementada el 04/10 — Muy bajo

2 Exportar estadísticas a CSV (Excel) · implementada el 04/10 — Bajo

3 Sincronización de partidos y equipos a Firestore · implementada el 23/10 · v1.3.0 — Medio + validación alta

4 Comparador de dos partidos · implementada el 04/10 — Bajo-medio

5 Panel de temporada · implementada — Medio-bajo

6 Fixture con autocompletado · implementada — Medio

7 Animar una jugada de pizarra · implementada — Medio

8 Backup en la nube (jugadas y fixture) · re-estimada el 24/10 — Medio

9 Scouting fase 2 y carga precisa (Etapa 3) — Alto

10 Chatbot conversacional (IA) — Muy alto

C Orden sugerido de implementación

A. Qué cambió desde el roadmap del 30/08

Varias funcionalidades del roadmap ya se implementaron desde entonces, algunas de una forma más simple que la que se proponía. No entran en la comparación de este documento:

Funcionalidad del roadmap	Estado	Cómo quedó
§2 Scouting de rival, fase 1	Hecho	`scouting_engine.dart` (`buildReport`) + `rival_scouting_screen.dart`, con botón propio en el Home. **Hoy no está gateado a premium**: falta decidir si queda así.
§3 Estadísticas por rotación	Hecho	`StatsEngine.computeRotations` reproduce la rotación de cada rally con `_replayRotations`. No hizo falta agregar `rotationSlot` a `RallyEvent` ni tocar `MatchController`, y funciona también con partidos viejos.
§8 Gráfico de progresión del marcador	Hecho	`computeTimelines` + `score_timeline_chart.dart`, con gráficos propios en `lib/widgets/charts/` (sin sumar `fl_chart`).
§8 Reportes PDF personalizables	Parcial	Con premium ya se eligen qué gráficos y mapas van al PDF (`VisualStatsPreferences`). Falta elegir secciones o columnas de la tabla de jugadores.

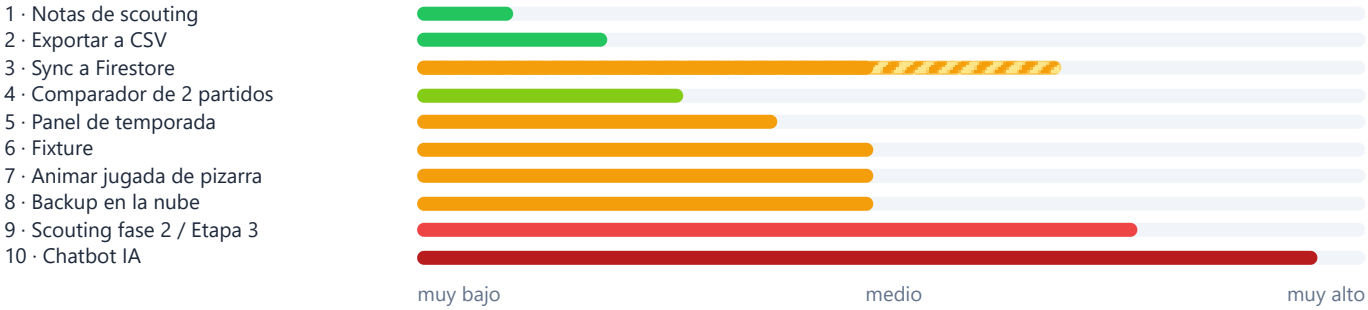
Las Etapas 1 y 2 de la estadística visual (pestañas "Mapas" y "Gráficos") están publicadas desde la versión 1.2.0. La Etapa 3 quedó pospuesta y aparece acá como parte de la funcionalidad 9.

B. Tabla comparativa

Ordenada de la más fácil a la más difícil según la revisión del 03/10 (el 3 y el 8 se re-estimaron después, ver la nota al pie; del 1 al 7 ya están hechas). "Carga en vivo" indica si toca `MatchController` o la captura de jugadas, que es la parte más delicada de la app; "Backend" indica si requiere cambios en el repo `volley_stats_app_backend`.

#	Funcionalidad	Qué ya existe y se reusa	Qué falta construir	Carga en vivo / Backend	Esfuerzo
1	**Notas de scouting por partido**	El patrón de campo opcional con migración `null` en `VolleyMatch`	Campo `notes` + `toJson` / `fromJson` + un `TextField` en el resumen	No / No	Muy bajo Hecho 04/10
2	**Exportar a CSV**	`match_export_service.dart` (share en mobile, `FilePicker` en desktop) y `MatchStats.orderedRows`	Un servicio nuevo de unas 80 líneas + un botón al lado del PDF	No / No	Bajo Hecho 04/10
3	**Sync de partidos y equipos a Firestore**	`cloud_firestore` ya estaba en el pubspec; `account_devices` sirvió de modelo para la regla por `uid`	Hecho: Etapa 1 (Hive por cuenta) y Etapa 2 (sync, papelera, política 2.0)	No / Sí, solo la regla	Medio validación alta Hecho 23/10 · v1.3.0
4	**Comparador de 2 partidos**	`StatsEngine.compute()` tal cual	Pantalla de selección + vista en dos columnas. Sacar `_StatsTable` a un widget compartido	No / No	Bajo-medio Hecho 04/10
5	**Panel de temporada**	`scouting_engine.dart` ya resuelve la agregación de varios partidos	Agregar por `playerId`, hacer públicos `_accumulate` / `_accumulateRecepcion`, filtros y reusar la tabla del punto 4	No / No	Medio-bajo Hecho
6	**Fixture con autocompletado**	`pickMode` de `PlaysArchiveScreen`, el patrón de Hive box y el combo de rival de `NewMatchScreen`	Modelo nuevo, box nueva, 2 pantallas, `fixtureEntryId` en `VolleyMatch` y un tile en el Home	No / No	Medio Hecho
7	**Animar jugada de pizarra**	`Play.strokes` ya guarda el orden de los trazos	Animación en `WhiteboardPainter` + controles de reproducción	No / No	Medio Hecho
8	**Backup en la nube**	`MatchSyncService` (push, pull, papelera, subida inicial) y el botón "Hacer una copia ahora"	Sumar jugadas y fixture al mismo sync; las fotos quedan afuera	No / Sí, solo la regla	Medio (antes Alto)
9	**Scouting fase 2 / Etapa 3**	Mapas y coordenadas de la Etapa 2, `zone_heatmap.dart`	Cargar la zona del rival o el toque preciso en vivo, con campos nuevos en `RallyEvent`	**Sí** / No	Alto
10	**Chatbot IA**	El backend ya tiene estructura de Cloud Functions con secretos (`redeemPromoCode`)	Proxy, armado del contexto, anonimización, límites de costo y UI	No / Sí, función nueva	Muy alto

Esfuerzo relativo



Escala cualitativa, no en días. En el sync (3), lo rayado fue la validación: actualizar encima con datos reales, dos dispositivos y modo avión (hecha el 23/10). El 8 bajó de Alto a Medio el 24/10: con el 3 hecho, sumar jugadas y fixture reusa todo el mecanismo del sync.

1. Notas de scouting por partido

Implementada el 04/10/2026 · v1.2.0

Muy bajo | Carga en vivo: no | Backend: no | Datos salen del dispositivo: no

Objetivo. Que el cuerpo técnico pueda dejar observaciones en texto libre sobre cada partido (cómo saca el rival, qué rotación nos costó, qué reforzar) y volver a leerlas antes del próximo cruce con ese rival.

QUÉ YA EXISTE

- El patrón de campo opcional nuevo: `null` en partidos viejos, como ya se hizo con `targetZone` y `missType` en `RallyEvent`.
- `AppDataController.saveMatch` para persistir. Al ser un partido que ya existe no pasa por el tope de 3.
- La pantalla de scouting, que ya lista los partidos contra cada rival.

QUÉ HAY QUE CONSTRUIR

- `volley_match.dart`: `String? notes` + `toJson` / `fromJson`.
- `match_summary_screen.dart`: tarjeta "Notas" en la pestaña Tabla, con un diálogo de edición multilínea.
- Opcional: mostrar las notas en `rival_scouting_screen.dart` debajo de cada partido, y como sección del PDF.



Boceto. Tarjeta de notas en el resumen del partido, el diálogo de edición y (opcional) las notas en el scouting del rival.

Gating y decisiones

- **¿Gratis o premium?** Por sí sola mueve poco la aguja como argumento de venta. Recomendación: dejar la edición gratis y que las notas se lean dentro del scouting (si el scouting pasa a ser premium, el valor queda del lado pago). Si se prefiere gatear, alcanza con envolver el ✎ en `runIfPremium`.
- Si se implementa el sync (3), las notas viajan solas dentro de `toJson()`; editarlas vuelve a subir el partido porque ya está terminado. No hay que hacer nada extra.

2. Exportar estadísticas a CSV (Excel)

Implementada el 04/10/2026 · v1.2.0

Bajo | Carga en vivo: no | Backend: no | Datos salen del dispositivo: solo si el usuario lo comparte

Objetivo. Que el cuerpo técnico se lleve la tabla de estadísticas a una planilla propia (Excel, Google Sheets) para cruzarla con sus datos, además del PDF de solo lectura que ya existe.

QUÉ YA EXISTE

- `MatchExportService`: hoja de compartir nativa en Android/iOS y `FilePicker.saveFile` en desktop. El CSV usa exactamente el mismo esqueleto.
- `MatchStats.orderedRows` y las 33 columnas de `_StatsTable` (N°, Jugador, Pts, Err, Saq PP… Rec %, Am, Ro).
- El selector de set del resumen: se exporta lo mismo que se está viendo (`StatsEngine.compute(match, setNumber: …)`).

QUÉ HAY QUE CONSTRUIR

- Nuevo `lib/services/csv_export_service.dart`: arma las filas a mano (sin paquete nuevo), una por jugador + "Total Equipo".
- `match_summary_screen.dart`: un `IconButton` nuevo al lado del PDF, envuelto en `runIfPremium`.
- Más adelante: reutilizar el mismo servicio para el scouting y el panel de temporada.



Boceto. Botón nuevo en la barra del resumen, hoja de compartir nativa y el archivo abierto en Excel, con las mismas columnas que la tabla de la app.

> **Corrección al roadmap: el gating no viene incluido.** El roadmap asumía que alcanza con que la pantalla contenedora esté gateada, pero `MatchSummaryScreen` deja que un usuario free vea un partido (`freeStatsMatchId`). El botón de CSV tiene que chequear premium por su cuenta con `runIfPremium` (`lib/widgets/premium_gate.dart`).

Detalles para que abra bien en Excel en español

- **Separador `;` en vez de `,`.** Excel configurado en español usa `;` como separador de listas; con comas, todo queda amontonado en la columna A. Los nombres que tengan `;` o comillas van entre comillas.
- **BOM UTF-8** (`\uFEFF`) al principio del archivo. Sin él, Excel rompe los acentos y la ñ de los nombres.
- **Porcentajes como enteros** (`67` en la columna "Saq %"), para evitar el problema de coma o punto decimal.

3. Sincronización de partidos y equipos a Firestore

Medio · Validación alta · Carga en vivo: no · Backend: regla nueva · Datos salen del dispositivo: sí, si el usuario lo activa

Implementada el 23/10/2026 · v1.3.0

> **Implementada y probada el 23/10/2026**: las Etapas 1 y 2 salen juntas en la versión 1.3.0+18 (no en versiones separadas, como se planeaba). Esta sección conserva el análisis con el que se decidió (revisión del 22/10, ampliada el 23/10 con los equipos). Lo que cambió al implementarla, la prueba con dos dispositivos y lo que se sumó después están en **"3 · Cómo quedó"**, al final de la sección. El requisito que mandaba sobre todo lo demás se cumplió: **no se pierde ningún partido ya archivado**.

Objetivo. Que el archivo de partidos y los equipos sean por **cuenta** y no por dispositivo: un partido terminado en la tablet aparece en el celular de la misma cuenta, y el equipo cargado en uno se puede usar en el otro para armar un partido nuevo. La carga en vivo, el archivo y el scouting siguen leyendo de Hive, 100% offline.



Diagrama. Solo viajan partidos terminados y solo con la nube activada. Un borrado viaja como marca (`deletedAt`) y manda el partido a la papelera de cada dispositivo. El pull avanza con la hora del servidor; los conflictos se resuelven con la hora de la edición.

El caso típico: cargar en el teléfono y abrir en otro dispositivo

ANTES (HASTA LA 1.2.0)

No se veía. Todo vivía en el Hive del teléfono; para pasarlo había que usar Exportar/Importar a mano.

DESDE LA 1.3.0

Se ve, si se cumplen tres condiciones:

1. El partido está **terminado** (los partidos en curso siguen pasando por Exportar/Importar).
2. La cuenta tiene **la nube activada**.
3. El teléfono **tuvo señal antes de cerrar sesión**: la cola offline de Firestore está atada a la cuenta; si se cierra sesión antes, el partido sale recién cuando esa cuenta vuelva a entrar en ese teléfono con conexión.

Decisiones

Tema	Decisión	Por qué
Qué se sube	Partidos terminados y equipos con sus jugadores (sin fotos), con un interruptor por cuenta, no una elección partido por partido	Un entrenador casi nunca quiere elegir partido por partido, y esa elección genera casos confusos entre dispositivos. Se puede sumar después sin rehacer nada.
Consentimiento	Apagado hasta que el usuario acepta un aviso único	Los planteles llevan fechas de nacimiento, posiblemente de menores. Hasta aceptar, todo sigue 100% local, como prometía la política 1.0.
Free o premium	Para todos (fijado como decisión en el prompt 3B)	Es el respaldo de lo que el usuario ya tiene; con 3 partidos el costo es mínimo. El valor pago sigue en el archivo ilimitado y los dispositivos simultáneos.

Tema	Decisión	Por qué
Tope de 3	Por cuenta. No aplica al bajar ni en la subida inicial; sí al crear un partido y al restaurarlo de la papelera	Restaurar sin tope permitiría pasarlo borrando y restaurando. Una cuenta free nueva en el mismo teléfono arranca con el archivo vacío (aceptado: coincide con el plan de que el límite sea por cuenta).
Conflictos	Gana la última edición (`updatedAt`)	En un partido terminado solo se editan las notas. El cursor del pull va aparte (`serverUpdatedAt`) para no depender del reloj de cada dispositivo.
Borrar	Papelera de 30 días, en todos los dispositivos	Un borrado por error se puede deshacer desde cualquier dispositivo.
Borrar la cuenta	Etapa 1: antes de confirmar se ofrece **Exportar todo**; los datos locales quedan en disco pero sin acceso. Etapa 2: se borran `matches/{uid}` y los cajones de esa cuenta en ese dispositivo. En los otros dispositivos solo se cierra la sesión	Hoy el diálogo promete que los datos quedan en el dispositivo. Con cajones por cuenta eso deja de ser cierto: **Importar copia completa** los carga en otra cuenta. El diálogo avisa que en los otros dispositivos hay que exportar antes de borrar.
Fixture	Por cuenta desde la Etapa 1, sin sync	`fixture_box` ya existe y sus filas apuntan a partidos de la cuenta (`linkedMatchId`). Si quedara por dispositivo, otra cuenta vería filas que apuntan a partidos que no tiene.
Equipos	Gana el equipo entero con el `updatedAt` más nuevo. Borrar un equipo lo oculta en todos los dispositivos y se purga a los 30 días. Las fotos no viajan	Sin los equipos, en otro dispositivo se ven los partidos pero no se puede armar uno nuevo. Si el mismo equipo se edita sin señal en dos dispositivos, queda la última edición (aceptado). Los partidos no dependen del equipo: guardan su copia del plantel.
Cupo free de estadística	Uno por cuenta en todos sus dispositivos: `freeStatsMatchId` en `account_devices/{uid}` , con caché local. Registrarlo necesita conexión	Si quedara en `settings_box` , una cuenta free lo recuperaría al entrar en otro dispositivo o al reinstalar. Se escribe "solo si no existe": gana el primero.
Actualizar sin sesión	El primer uid que entra queda como dueño de los datos actuales, aunque sea otra persona	Aceptado: es poco probable y se corrige con Exportar todo / Importar copia completa.
Partido abierto	El pull no toca un partido abierto en la carga en vivo; lo aplica en el pull siguiente	Un partido en curso exportado a otro dispositivo y terminado allá vuelve con el mismo id; pisarlo con la carga abierta haría que el próximo guardado reescriba la versión vieja.

Dos etapas, en versiones separadas

ETAPA 1 · DATOS LOCALES POR CUENTA

Se puede publicar sola: arregla un bug actual

Hoy `teams_box`, `matches_box`, `plays_box` y `fixture_box` son uno por instalación y `signOut` no los toca: otra cuenta en el mismo teléfono ve el archivo de la anterior. Con el sync, eso mezclaría partidos entre cuentas.

- **Los cajones actuales no se mueven:** pasan a ser de la cuenta dueña (`legacy_owner_uid` en `settings_box`).
- Las demás cuentas estrenan `matches_box_<sufijo>`, etc. El sufijo sale del uid sin depender de mayúsculas, porque Hive pasa los nombres a minúsculas.
- Cupo free de estadística por cuenta en todos sus dispositivos (`account_devices/{uid}`) y caché de la suscripción por cuenta. `settings_box` sigue siendo del dispositivo.
- Copia JSON de los 4 cajones antes del primer cambio, fuera del Auto Backup.
- `loadAll()` pasa a correr después del login. Al cerrar sesión, primero se vuelve a la primera pantalla y después se cierran los cajones.
- Exportar todo / Importar copia completa / Restaurar copia automática.

ETAPA 2 · SINCRONIZACIÓN

Requiere la Etapa 1 y la política de privacidad nueva

- Nuevo `match_sync_service.dart`: push, subida inicial reanudable y pull incremental de equipos y partidos.
- `updatedAt` y `deletedAt` opcionales en `VolleyMatch` y `Team` (`null` en datos viejos).
- Papelera de 30 días: los partidos borrados salen de `AppDataController.matches`, así que quedan fuera del archivo, el scouting, la temporada, el impacto, el comparador, el fixture y el tope sin tocar cada pantalla.
- Aviso único + switch "Partidos y equipos en la nube" en Cuenta.
- `deleteAccount` borra `matches/{uid}`, `teams/{uid}` y los cajones de la cuenta.
- Política de privacidad, sección Seguridad de los datos de Play y regla en `firestore.rules`.

Lo que ya está en el código y juega a favor

Qué hay	Para qué sirve
`cloud_firestore` en el pubspec, usado en Android y Windows para `account_devices`	No hay paquete nuevo y Firestore ya funciona donde funciona el login. Falta confirmar la cola offline en Windows; si no la hay, ahí el interruptor no aparece.
Campos opcionales en `VolleyMatch` (`notes`, `fixtureEntryId`)	Mismo patrón para `updatedAt` y `deletedAt`, sin migración.
Reglas de Auto Backup que solo excluyen la sesión de Firebase Auth	Los cajones nuevos se respaldan solos. Solo hay que excluir la carpeta de la copia previa.
`account_devices/{uid}` y `deleteAccount`, que lo borra antes que al usuario	Lugar para el campo `cloudSync`, modelo para la regla por `uid`, y el orden que necesita borrar `matches/{uid}` (las reglas exigen seguir autenticado).

Garantías para no perder partidos

Riesgo	Cómo se evita
La migración a cajones por cuenta se corta a la mitad o falla	**No hay migración:** los cajones actuales quedan tal cual y solo se anota su dueño.
Algo sale mal en la primera versión con este cambio	Copia JSON automática de equipos, partidos, jugadas y fixture antes de tocar nada. Se restaura desde la app ("Restaurar copia automática") y queda fuera del Auto Backup para no pasar su tope de 25 MB.
Se cierra la sesión con la carga en vivo abierta (cuenta borrada desde otro dispositivo, autoexpulsión por baja de plan)	Se vuelve a la primera pantalla antes de cerrar los cajones. El partido ya está guardado acción por acción y se retoma al volver a entrar con esa cuenta.
Se borra la cuenta	Antes de confirmar se ofrece Exportar todo; Importar copia completa lo carga en otra cuenta.

Riesgo	Cómo se evita
Los partidos ya archivados nunca suben (no vuelven a pasar por `saveMatch`)	**Subida inicial** de todo el archivo al activar la nube, reanudable y sin tope de 3.
Un partido no entra en un documento de Firestore (1 MiB como máximo, sin listas dentro de listas)	Se mide antes de implementar. Las listas anidadas se convierten solo en el JSON de Firestore; el partido que no entra no se sube y se avisa en Cuenta.
Un borrado por error se propaga a todos los dispositivos	Papelera de 30 días, restaurable desde cualquiera.
Un error de red o de reglas hace "desaparecer" partidos de Firestore	La bajada nunca borra un partido local porque falte en la nube: solo actúa ante una marca `deletedAt` explícita.
Un equipo se edita sin señal en dos dispositivos	Queda la última edición del equipo entero (aceptado). Ningún partido cambia: cada uno guarda su plantel.
Relojes distintos entre dispositivos	El pull avanza con `serverUpdatedAt` (hora del servidor) y con un margen hacia atrás; aplicar dos veces el mismo documento no cambia nada.
Cerrar sesión, apagar la nube o entrar con otra cuenta	Ninguno toca los datos locales. Push, pull y subida inicial se cortan y nunca escriben en los cajones de otra cuenta. Apagar la nube pregunta aparte si borrar la copia remota (por defecto, no).
Dos cuentas ya usaron el mismo teléfono antes de esta versión	**Límite aceptado:** sus datos están mezclados y quedan con la cuenta dueña. No se pierden; se pasan con Exportar todo / Importar copia completa.

Qué cambió desde el 14/10.

- Los equipos con sus jugadores pasan al sync (23/10); las jugadas y el fixture siguen en el backup manual (8).
- `fixture_box` ya existe (el 6 se hizo antes que el 3): entra en la Etapa 1 y en la copia previa.
- El cupo free de estadística sigue sin liberarse al borrar el partido (el prompt anterior lo liberaba) y pasa a ser uno por cuenta en todos sus dispositivos. La caché de la suscripción también es por cuenta.
- Cierre de sesión forzado con pantallas abiertas, y borrar cuenta con Exportar todo / Importar copia completa.
- La papelera se resuelve en `AppDataController.matches` (con 5, 6 y 7 hay más pantallas que leen el archivo) y restaurar respeta el tope.
- Límites de Firestore (1 MiB, listas anidadas), cursor con `serverUpdatedAt`, el pull no toca un partido abierto y la copia previa queda fuera del Auto Backup.

Cómo probarlo

- **Actualizar encima:** instalar el APK publicado con partidos, equipos, jugadas y fixture reales y actualizar sin desinstalar, con sesión abierta, sin sesión y con una segunda cuenta. El archivo tiene que quedar completo.
- **Cierre forzado:** borrar la cuenta desde otro dispositivo mientras este está en la carga en vivo. No tiene que romperse ni guardar nada en otro lado.
- **Borrar cuenta:** Exportar todo, borrar, entrar con otra cuenta e Importar copia completa.
- **Cupo free:** gastarlo en un dispositivo y verificar que en otro, o después de reinstalar, ya está gastado.
- **Dos dispositivos:** terminar un partido en uno y verlo en el otro; borrarlo, verlo en la papelera de ambos y restaurarlo. Editar un equipo en uno y armar un partido con él en el otro.
- **Modo avión:** terminar sin señal, reconectar y verificar que llega solo.
- **Sin tocar datos reales:** Firebase Emulator Suite o una cuenta de prueba descartable.
- **Plan completo:** el doc *RallyStats · Plan de pruebas del sync (3A y 3B)* tiene todos los casos con pasos, resultado esperado y estado.

3 · Cómo quedó (23 y 24/10/2026)

Versión 1.3.0+18

Resumen de lo que se implementó en las dos sesiones del 23/10 (Etapa 1 y Etapa 2), de lo que se decidió en el camino y de lo que se sumó el 23 y el 24/10. El detalle técnico está en el `CLAUDE.md` de la app (Arquitectura → Datos locales por cuenta, y Sync con la nube y papelera).

ETAPA 1 · DATOS LOCALES POR CUENTA

- Cajones de Hive por cuenta, con dueño legacy y sin migración (`openUserBoxes`, sufijo en base32).
- Copia previa automática de los 4 cajones, fuera del Auto Backup.
- Exportar todo, Importar copia completa y Restaurar copia automática.
- Cupo free de estadística y caché de la suscripción por cuenta.
- Cierre de sesión forzado con pantallas abiertas sin guardar en otra cuenta.
- **Prueba en dispositivo:** 11 pasos aprobados el 23/10 (actualizar encima con y sin sesión, segunda cuenta, cupo free, cierre forzado, borrar cuenta).

ETAPA 2 · SYNC Y PAPELERA

- `match_sync_service.dart`: push, subida inicial reanudable y pull incremental de equipos y partidos.
- Papelera de 30 días con restaurar (respeta el tope en free) y eliminar definitivamente; purga local y remota.
- Aviso único después del login + switch "Partidos y equipos en la nube" en Configuración.
- Borrar cuenta borra `matches/{uid}`, `teams/{uid}` y los cajones locales.
- Política de privacidad 2.0 y Términos 1.1; reglas de Firestore desplegadas el 23/10.
- 30 tests nuevos sin red (fake de `CloudSyncStore`).

Los dos límites de Firestore, medidos antes de diseñar

Límite	Resultado
Sin listas dentro de listas	No hay ninguna: las listas de eventos están dentro del mapa de cada set (lista → mapa → lista). Firestore las acepta sin convertir; solo las fechas pasan a `Timestamp`.
1 MiB por documento	40 partidos de 5 sets simulados, con 9 zonas, 16 jugadores y notas largas: **154 a 193 KB** (~395 bytes por evento). Para llegar a 700 KB haría falta 3,6 veces el más pesado. Igual, un partido de más de 700 KB no sube y se avisa en Configuración.

Decisiones tomadas al implementar (no estaban en el análisis)

Tema	Cómo quedó y por qué
Convergencia	Si en el pull lo local es más nuevo que la nube, se vuelve a subir. El push escribe sin mirar lo que hay: una escritura vieja encolada en otro dispositivo podía dejar la nube atrás sin que nadie la corrigiera.
Papelera y conflictos	Mandar a la papelera y restaurar también estampan `updatedAt` (si no, la marca llegaba empatada y no se aplicaba). Gana siempre la versión más nueva; si esa versión tiene `deletedAt`, va a la papelera.
Borrado que llega sin copia local	Entra directo a la papelera, para que la papelera sea la misma en todos los dispositivos.
Eliminar definitivamente	Borra local y nube. En los otros dispositivos sigue en su papelera hasta su purga: que falte en la nube nunca borra nada.
Abandonar partido	También va a la papelera (antes se borraba sin vuelta atrás).
Exportar todo	Incluye la papelera y los equipos borrados; importar no los duplica.
Equipo de ejemplo	Ids fijos y no se sube hasta que se edita: cargarlo en dos dispositivos deja un solo documento.
Consentimiento	La política 2.0 no se re-pide a las cuentas existentes: al activar la nube se guarda `cloudConsent` con la versión aceptada.
Regla opcional	Rechazar en `firestore.rules` un `updatedAt` más viejo que el guardado: ofrecida, no aplicada (la convergencia la resuelve la app).

Prueba con dos dispositivos (23/10/2026)

Con dos celulares (Principal y Viejo), una cuenta free descartable y alternando la sesión (la cuenta tiene un solo lugar). Todos los pasos aprobados:

#	Paso	Qué se comprobó	Estado
0	Reglas de Firestore	Desplegadas sin tocar `account_devices` ni `promo_codes`	OK
1	Instalar y cargar datos	El aviso aparece con la cuenta real y con la nueva; "Ahora no" guarda `cloudSync: false`	OK
2	Activar desde Configuración	Subida inicial, política 2.0 aceptada, `photoPath` en null en la nube	OK
3	Segundo dispositivo	Baja equipos (sin fotos) y partidos terminados; el en curso no viaja	OK
4	Partido nuevo y tope	El cuarto partido baja aunque pase el tope; crear uno nuevo sigue frenado	OK
5	Papelera	Borrado visible en los dos, restaurar frenado por el tope en uno y permitido en el otro	OK
6	Editar un equipo	Viaja en las dos direcciones y la foto local se conserva	OK
7	Modo avión	Terminado sin señal, con la app cerrada en el medio: llega solo al reconectar	OK
8	Apagar la nube	Sin borrar la copia y borrándola; nunca toca lo local; el interruptor es por cuenta	OK
9	Eliminar la cuenta	Borra la nube, `account_devices` y el usuario; la cuenta real queda intacta	OK
10	Partido abierto en la carga en vivo	Necesita dos lugares de dispositivo a la vez: queda cubierto por un test automático	Solo test

Lo que se sumó el 23 y el 24/10 (fuera del plan)

Qué	Detalle
Login al primer intento	Tras cerrar sesión y volver a entrar, la escucha de `account_devices/{uid}` leía primero la caché vieja y cerraba la sesión sin mensaje. Ahora solo actúa con datos del servidor (`AuthService.slotLost`). Confirmado en dispositivo.
"Hacer una copia ahora"	Debajo del interruptor de la nube: vuelve a subir todo en el momento, aunque ya se suba solo. Sin red avisa enseguida.
Manual en la app Premium	Configuración → "Manual de usuario": baja el PDF publicado en Firebase Hosting y lo muestra dentro de la app (compartir, imprimir; sin red, la última copia). **Regla del release: cada vez que se regenera el manual, `firebase deploy --only hosting` .**
Manual y FAQs	Secciones nuevas 2.3 (nube) y 14.4 (papelera), FAQs corregidas (abandonar, corregir estadísticas, tope) y una nueva (cambiar de celular).

Estado del release 1.3.0+18

LISTO

- APK, .aab, instalador y .zip de Windows compilados (24/10).
- Política 2.0 publicada en el link de siempre.
- Texto de "Seguridad de los datos" para Play en `documents/Play-Seguridad-de-los-datos.md` y notas de la versión redactadas.
- 191+ tests en verde y `flutter analyze` sin errores.

PENDIENTE

- Completar "Seguridad de los datos" en Play Console y subir el .aab.
- Publicar el manual: `firebase deploy --only hosting` desde `volley_stats_app` .
- Commit de las Etapas 1 y 2 en los dos repos.
- Validar con usuarios reales antes de retomar el 9 y el 10.

4. Comparador de dos partidos

Implementada el 04/10/2026 · v1.2.0

Bajo-medio | Carga en vivo: no | Backend: no | Datos salen del dispositivo: no

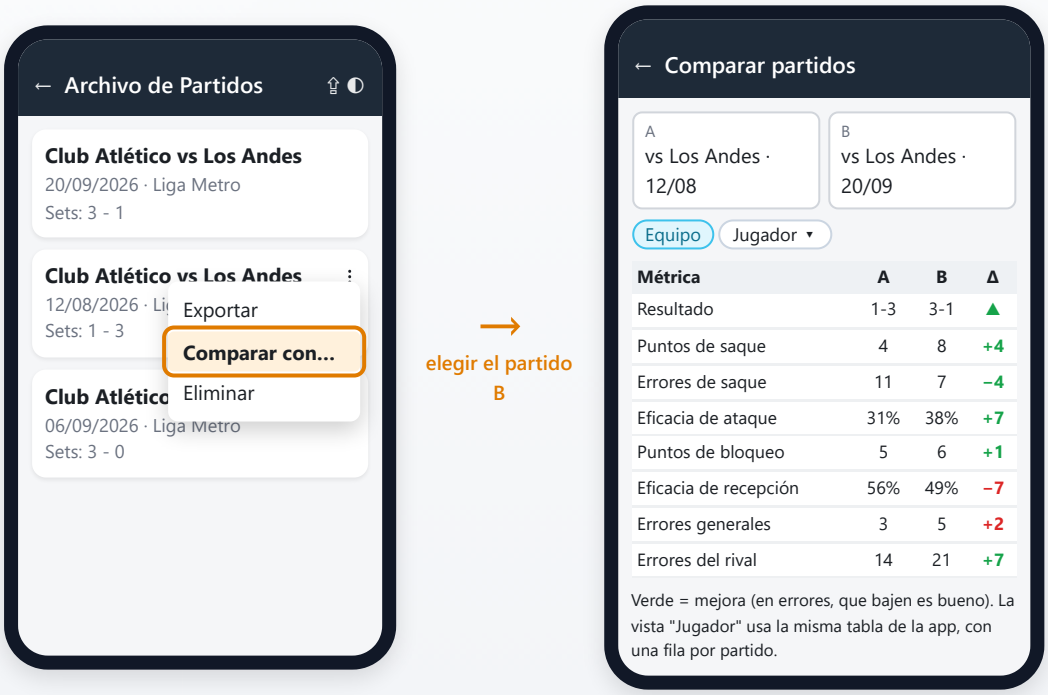
Objetivo. Poner dos partidos lado a lado, por ejemplo la ida y la vuelta contra el mismo rival, o un partido antes y otro después de cambiar el sistema de juego, y ver de un vistazo qué mejoró y qué empeoró.

QUÉ YA EXISTE

- `StatsEngine.compute()` se llama una vez por partido; no hace falta agregación nueva.
- `MatchStats` ya trae totales de equipo y errores y puntos del rival.
- El menú ⋮ de cada partido en el archivo (hoy "Exportar" y "Eliminar").

QUÉ HAY QUE CONSTRUIR

- **Refactor:** sacar `_StatsTable`, `_H` y `_C` de `match_summary_screen.dart` a `matches/widgets/stats_table.dart`, sin cambios visuales. Esa tabla también la usa el Panel de temporada.
- Nueva `matches/match_compare_screen.dart`: dos selectores, una vista "Equipo" y una vista "Jugador".
- Entrada "Comparar con..." en el menú ⋮, que deja preseleccionado el partido A.



Boceto. Entrada desde el menú ⋮ del archivo y comparación de equipo con diferencia coloreada.

Gating y riesgos

- **Premium natural:** `runIfPremium` en "Comparar con...". Un usuario free solo puede ver la estadística de un partido, así que comparar ya excede lo que tiene.
- **Jugadores entre partidos:** `ownRoster` es una copia por partido; el mismo jugador conserva su `Player.id` si los dos partidos se armaron desde el mismo `Team`. Si se cargó un plantel a mano, la vista "Jugador" no puede emparejar (la de equipo sí funciona). Conviene mostrar solo los jugadores presentes en los dos partidos.

5. Panel de temporada

Implementada

Medio-bajo | Carga en vivo: no | Backend: no | Datos salen del dispositivo: no

Objetivo. Ver la evolución de cada jugador y del equipo sumando todos los partidos guardados, con filtros por torneo y por rango de fechas. Hoy la estadística existe solo partido por partido.

Por qué ahora es más fácil que en el roadmap: `scouting_engine.dart` ya resuelve el problema de fondo, que es filtrar partidos, recorrerlos y sumar resultados de `StatsEngine` en un reporte. El panel de temporada es lo mismo pero agrupando por jugador propio en vez de por rival.



Diagrama y boceto. El motor de agregación reutiliza todo lo que ya calcula `StatsEngine`; la pantalla reutiliza la tabla del resumen y los gráficos de `lib/widgets/charts/`.

QUÉ HAY QUE CONSTRUIR

- Nuevo `lib/services/season_engine.dart` (calcado de `scouting_engine.dart`).
- Hacer públicos `_accumulate` / `_accumulateRecepcion`, o moverlos como métodos `addFrom()` en `stat_line.dart`, en vez de duplicarlos.
- Nueva `matches/season_stats_screen.dart` con filtros, indicadores y la tabla compartida.
- Entrada: tile "Mi temporada" en el Home con `runIfPremium`.

DECISIONES Y RIESGOS

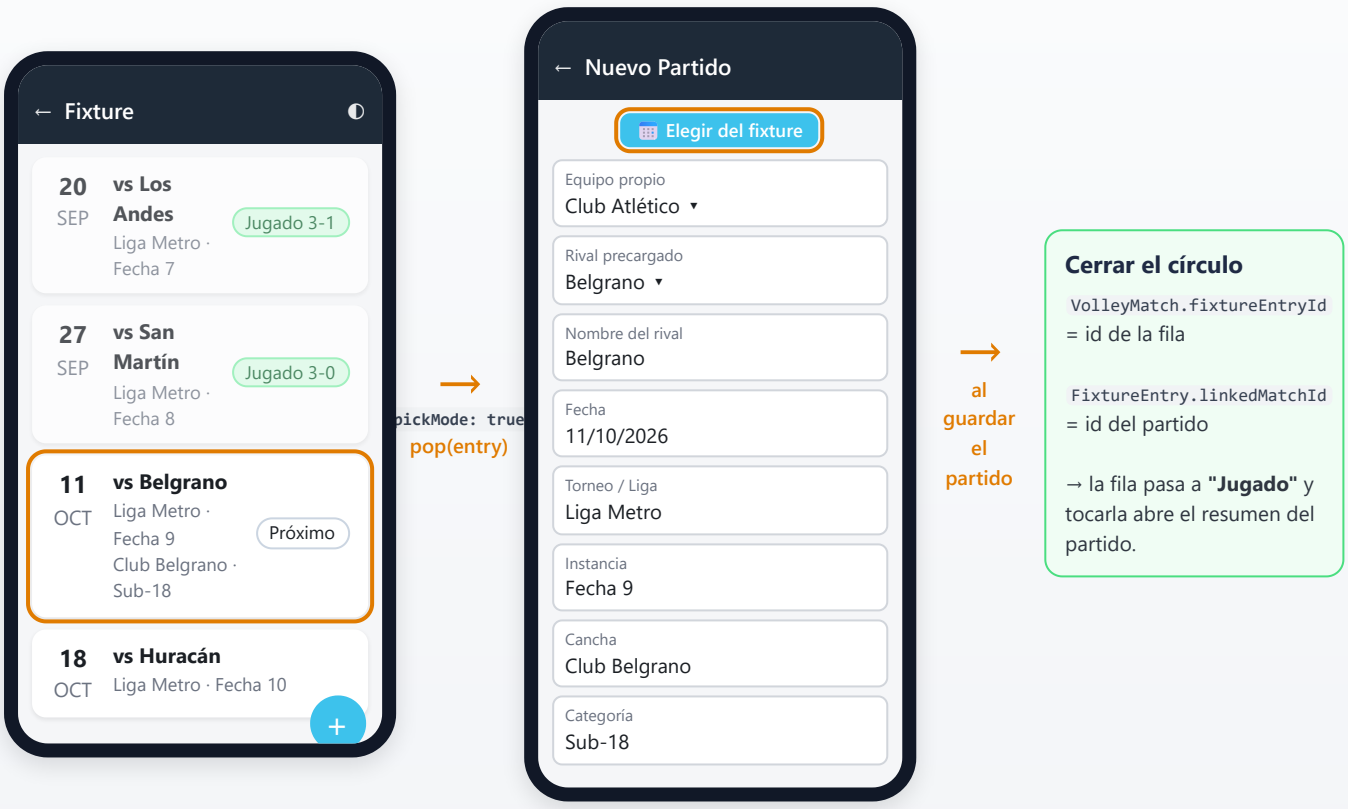
- **Premium natural:** con el tope de 3 partidos, un free casi no tiene temporada para ver.
- Nombre y dorsal: usar los del partido más reciente si cambiaron.
- Partidos sin `ownTeamSourceId` (equipo cargado a mano): agrupar por `ownTeamName` como alternativa.
- Con muchos partidos, calcular fuera del `build` (una vez por cambio de filtro).

6. Fixture con autocompletado

Implementada

Medio | Carga en vivo: no | Backend: no | Datos salen del dispositivo: no

Objetivo. Cargar el calendario del torneo (fecha, rival, instancia, cancha, categoría) y, al crear un partido, elegirlo del fixture para que los datos se completen solos. Además le da al usuario un motivo para abrir la app entre partido y partido.



Boceto. Lista del fixture, autocompletado en Nuevo Partido (campos resaltados) y vínculo entre la fila del fixture y el partido jugado.

QUÉ YA EXISTE

- `PlaysArchiveScreen(pickMode: true)`: el mismo mecanismo de "elegir y devolver con `Navigator.pop`".
- El patrón de Hive box (`teams_box`, `matches_box`, `plays_box`) y de `toJson` / `fromJson` manual.
- `NewMatchScreen` ya autocompleta `_rivalNameCtrl` al elegir un `Team` rival; el fixture es otra fuente para los mismos controles.

QUÉ HAY QUE CONSTRUIR

- `models/fixture_entry.dart`; `fixture_box` en `storage_service.dart`; `fixtureEntries` + alta, baja y edición en `AppDataController`.
- `screens/fixture/fixture_screen.dart` y `fixture_entry_form_screen.dart`.
- `new_match_screen.dart`: botón + `_pickFromFixture()`.
- `fixtureEntryId` opcional en `VolleyMatch` y tile nuevo en el Home.

Decisiones tomadas

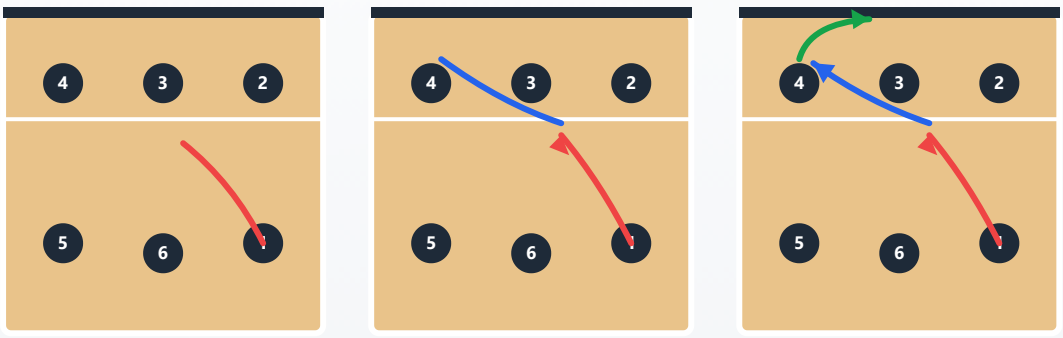
- **¿Gratis o premium?** Gratis y sin tope: es una función de organización y sirve como gancho de uso frecuente.
- **¿Se sincroniza?** No, queda local. Con la Etapa 1 del sync (3) pasa a ser por cuenta, igual que los partidos, y entra en el backup manual (8).

7. Animar una jugada de pizarra

Implementada

Medio | Carga en vivo: no | Backend: no | Datos salen del dispositivo: no

Objetivo. Reproducir una jugada guardada dibujándose trazo por trazo, como si el entrenador la estuviera dibujando en ese momento, para explicársela al equipo en el banco o en la práctica.



Boceto. Tres momentos de la reproducción (saque, armado, ataque) y la barra de controles. Las fichas quedan fijas desde el inicio y se animan los trazos en el orden en que se dibujaron.

QUÉ YA EXISTE

- `Play.strokes`: cada `PlayStroke` guarda `pointsX` / `pointsY` en el orden en que se dibujaron, con su color y si termina en flecha.
- `WhiteboardPainter` ya dibuja trazos, flechas y fichas.
- `PlaysArchiveScreen` para elegir la jugada.

QUÉ HAY QUE CONSTRUIR

- `WhiteboardPainter` recibe un `progress` (0 a 1): dibuja los trazos completos hasta N y una parte del trazo N+1. La flecha aparece solo cuando su trazo termina.
- Modo "Reproducir" (pantalla nueva o modo de `whiteboard_screen.dart`) con un `AnimationController`, play/pausa, barra para adelantar o retroceder y velocidad.

Alcance y gating

- **Las fichas no se mueven:** `PlacedToken` guarda solo su posición final, no un recorrido. Animar el desplazamiento de jugadores requeriría grabar trayectorias, y eso queda fuera de esta funcionalidad.
- **Gating:** no hace falta nada nuevo, porque la pizarra ya está detrás de `runIfPremium` en el Home. Es una mejora de algo que ya es premium, más que un argumento de venta propio.
- Riesgo bajo: es solo interfaz y no toca el formato guardado de `Play`.

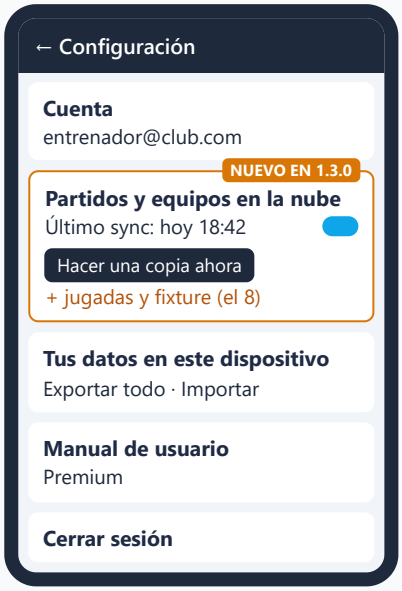
8. Backup en la nube (jugadas y fixture)

Re-estimada el 24/10/2026

Medio (antes Alto) · Carga en vivo: no · Backend: regla nueva · Datos salen del dispositivo: sí, si el usuario activa la nube

Objetivo. Que un entrenador no pierda nada si cambia de dispositivo o reinstala. **Con el sync (3) ya implementado en la 1.3.0**, los partidos terminados y los equipos ya quedan por cuenta y en la nube; lo que falta son las jugadas de la pizarra y el fixture.

Dato	¿Cubierto en la 1.3.0?
Partidos terminados	✓ **por el sync (3)**
Equipos y planteles (`teams_box`)	✓ **por el sync (3)**
Partidos en curso	No se suben (Exportar)
Jugadas de pizarra (`plays_box`)	✗ **falta**
Fixture (`fixture_box`)	✗ **falta**
Fotos de jugadores (`player_photos/`)	✗ **requeriría Storage**
Preferencias (tema, gráficos)	No hace falta



Boceto. Qué cubre ya la 1.3.0 y dónde entraría el 8: en el mismo interruptor y el mismo botón de copia, sin un segundo "backup" que confunda.

Dos caminos posibles · re-evaluados con el sync ya hecho

A · EXTENDER EL SYNC A JUGADAS Y FIXTURE

RECOMENDADO AHORA

- Dos colecciones más (`plays/{uid}/…` , `fixture/{uid}/…`) en `MatchSyncService` : push al guardar, pull, papelera y subida inicial ya existen y están probados.
- Mismo conflicto que los equipos: gana la última edición (`updatedAt`).
- El fixture apunta a partidos (`linkedMatchId`) que ya viajan: en otro dispositivo la fila figura como jugada sin hacer nada extra.
- Para el usuario es transparente: el mismo interruptor y el mismo botón.

B · BACKUP MANUAL (LO QUE SE RECOMENDABA ANTES)

- Botones "Hacer backup" y "Restaurar" con una foto completa de jugadas y fixture.
- Sin conflictos, pero suma un segundo mecanismo de nube al lado del sync, con su propio formato y sus propias pruebas.
- Para eso ya existe Exportar todo / Importar copia completa, que cubre todo (fotos incluidas no, igual que acá).

Riesgos

- **Fotos:** siguen afuera (necesitarían Storage). Al bajar, cada jugador conserva su foto local si existe, como ya hace el sync de equipos.
- **Tamaño:** una jugada son trazos con puntos; medir una jugada larga antes de implementar, como se hizo con el partido (límite de 1 MiB por documento).
- **Privacidad:** la política 2.0 dice que jugadas y fixture no se suben. Hacer el 8 obliga a una versión 2.1 de la política y a actualizar la ficha de Play.
- **Costo:** jugadas y fixture pesan poco al lado de los partidos; no cambia el orden de magnitud.

9. Scouting fase 2 y carga precisa (Etapa 3)

Alto | Carga en vivo: sí | Backend: no | Datos salen del dispositivo: no

Objetivo. Dos mejoras de la misma naturaleza, porque las dos **agregan datos que hoy no se capturan durante el partido**:

- **Scouting fase 2:** registrar a qué zona ataca el *rival* para armar su mapa de calor en el scouting. Hoy `computeZones()` y los mapas solo cubren al equipo propio.
- **Etapa 3 de la estadística visual** (pospuesta el 02/10): tocar el punto exacto de la cancha (`targetX`/`targetY`, origen opcional) en vez de elegir una zona, según `spec-estadistica-visual.md` §6.



Boceto. Paso opcional en el diálogo de punto del rival (con "Omitir" para no frenar la carga) y el mapa de calor resultante en el scouting.

QUÉ HAY QUE CONSTRUIR

- `rally_event.dart`: `rivalTargetZone` (fase 2) y/o `targetX/Y`, `originX/Y` (Etapa 3), opcionales y `null` en partidos viejos.
- `rival_action_dialog.dart` / `hit_zone_picker.dart` / `touch_dialog.dart`: el selector en vivo.
- `match_controller.dart` (`_addEvent`): guardar el dato nuevo.
- `scouting_engine.dart`: agregar zonas por rival y mostrarlas con `zone_heatmap.dart`.

POR QUÉ ES ALTO

- Toca el **corazón de la app** y el formato guardado de `RallyEvent`; requiere probar con partidos completos.
- Cada toque extra durante un partido en vivo compite con la velocidad de carga, que es la principal virtud de la app. El paso tiene que ser opcional y rápido.
- El valor llega con retraso: solo los partidos cargados *después* del release tienen el dato nuevo.

> **Recomendación:** retomarlo después de validar con usuarios reales las Etapas 1 y 2 (todavía sin publicar). Si se hace, conviene hacer fase 2 y Etapa 3 juntas, porque tocan los mismos archivos de la carga en vivo y conviene hacer una sola ronda de pruebas.

10. Chatbot conversacional (IA)

Muy alto | Carga en vivo: no | Backend: Cloud Function nueva | Datos salen del dispositivo: sí (anonimizados)

Objetivo. Hacer preguntas en lenguaje natural sobre el partido abierto ("¿quién recibió mejor en el set 3?", "¿por qué no me deja un tercer cambio de líbero?") y recibir respuestas redactadas a partir de datos que la app ya calculó. No hace falta entrenar un modelo propio: se usa un modelo existente vía API y se le pasa el contexto en cada consulta.



Diagrama y boceto. Versión acotada recomendada en el roadmap (opción A): preguntas sobre el partido abierto y sobre el manual, con nombres anonimizados y un tope mensual.

QUÉ YA EXISTE

- En el backend, `redeemPromoCode.ts` ya es un `onCall` con secreto (`defineSecret`) y verificación de entitlements: sirve de molde para el proxy.
- `StatsEngine` y `visual_stats.dart` dan el contexto ya calculado.
- `tool/generate_manual.dart` : el texto del manual para las preguntas de reglas y de uso.

QUÉ HAY QUE CONSTRUIR

- `functions/src/askCoach.ts` : auth, premium, cupo, llamada a la API y registro de uso.
- Cliente: el armado del contexto con anonimización y re-mapeo de nombres.
- `lib/screens/chatbot/` : pantalla con historial, estados de carga y error, y mensaje sin conexión.
- Política de privacidad, ficha de Play y tope de costo.

Por qué es el más grande: es la única con **costo variable por uso**, suma una tercera pieza de infraestructura (además de Firebase y RevenueCat) y obliga a acordar antes cómo manejar alucinaciones y datos de menores.

C. Orden sugerido de implementación

No es un compromiso, es una propuesta según esfuerzo, riesgo y dependencias entre funcionalidades. Actualizado el 24/10/2026.

Tanda 1 Hecha

Implementada el 04/10/2026 · versión 1.2.0

- **1** Notas de scouting
- **2** Exportar a CSV
- **4** Comparador de 2 partidos

Nada tocó la carga en vivo ni el backend. Dejó hecha la tabla compartida que usa la tanda 2.

Tanda 2 Hecha

Reporte multi-partido + sync · versión 1.3.0

- **5** Panel de temporada
- **3** Etapa 1: datos locales por cuenta
- **3** Etapa 2: sync, papelera y política 2.0

Las dos etapas del 3 se implementaron y probaron el 23/10 y salen juntas en la 1.3.0, no en versiones separadas.

Tanda 3 Hecha

Organización y pizarra

- **6** Fixture
- **7** Animar jugada de pizarra

Se hizo antes que el 3. El fixture quedó gratis, sin tope y local; con la Etapa 1 es por cuenta.

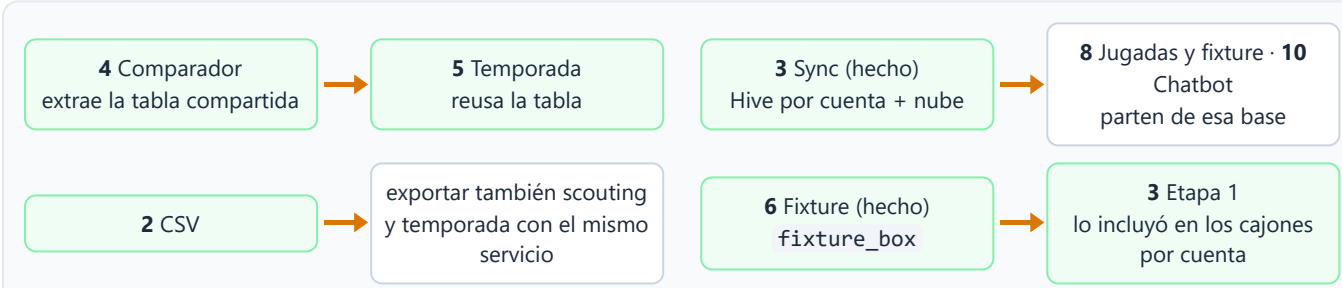
Más adelante

Conversar el alcance primero

- **8** Jugadas y fixture en la nube (bajó a Medio)
- **9** Scouting fase 2 / Etapa 3
- **10** Chatbot IA

Conviene esperar feedback real de la 1.3.0. El 8 es el siguiente natural: reusa el sync.

Dependencias entre funcionalidades



Decisiones de producto

Decisión	Afecta a	Estado
¿El scouting de rivales (ya hecho) pasa a ser premium?	Scouting, Notas (1)	**Abierta** Hoy está libre. Si se gatea, las notas suman valor del lado pago.
¿El fixture es gratis, premium o tiene un tope?	Fixture (6)	**Decidida** Gratis, sin tope y local.
¿La nube de partidos es gratis o premium?	Sync (3)	**Decidida e implementada** Gratis: es el respaldo de lo que el usuario ya tiene.
¿Qué datos se sincronizan además de los partidos?	Sync (3), Fixture (6), Backup (8)	**Decidida e implementada** Partidos terminados y equipos, sin fotos. Jugadas y fixture: propuesta de sumarlos al sync en el 8.
¿Qué pasa con los datos locales al borrar la cuenta?	Sync (3)	**Implementada** Se ofrece Exportar todo; se borran la nube y los cajones de la cuenta en ese dispositivo; en los otros se cierra la sesión.
¿El cupo free de estadística es por dispositivo o por cuenta?	Estadística free, Sync (3)	**Implementada** Por cuenta, en todos sus dispositivos.
Texto nuevo de la política de privacidad y de la ficha de Play	Sync (3), Backup (8), Chatbot (10)	**Hecha** Política 2.0 y Términos 1.1 (23/10); texto de Play listo. El 8 y el 10 van a pedir una versión nueva.
¿El manual dentro de la app es premium? ¿Dónde se publica?	Configuración	**Implementada** Premium. Firebase Hosting; se publica con `firebase deploy --only hosting` cada vez que cambia.